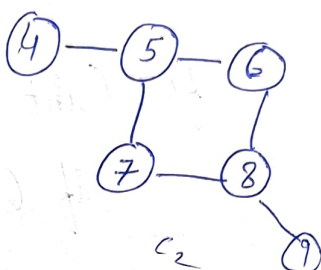


lec 88

Cycle Detection in Undirected graph.

① using BFS (Queue)

Graph :



Adj list

1 → 2

2 → 1, 3

3 → 2

4 → 5

5 → 6, 7

6 → 8, 5

7 → 5, 8

8 → 6, 7, 9

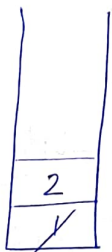
9 → 8

visited map

1	→ TRUE
2	→
3	→
4	→
5	→
6	→
7	→
8	→
9	→

Parent

u	→ v
1	→ 4
2	→ 1



queue (BFS)

when node is visited & that node is ~~node~~ not parent then cycle is present.

code

```
bool isCyclic (int src,
unordered_map <int, bool> &visited,
unordered_map <int, list <int>>
&adj.) {
```

```
    unordered_map <int, int> parent;
    parent[src] = -1;
    visited[src] = 1;
    queue <int> q;
    q.push (src);
```

```
    while (!q.empty()) {
        int front = q.front();
        q.pop();
```

```
        for (auto neighbour : adj[front]) {
            if (visited[neighbour] ==
                true && neighbour != parent
                    [front]) {
                return true;
            }
```

```
            else if (!visited[neighbour]) {
                q.push(neighbour);
                visited[neighbour] = 1;
                parent[neighbour] = front;
            }
        }
    }
```

```
    return false;
}
```

```
string cycleDetection (vector
<vector <int>> & edges, int n,
int m) { // create map (adj list)
    unordered_map <int, list <int>
        adj;
```

```
    for (int i=0; i<m; i++) {
        int u = edges[i][0];
        int v = edges[i][1];
        adj[u].push-back(v);
        adj[v].push-back(u);
    }
```

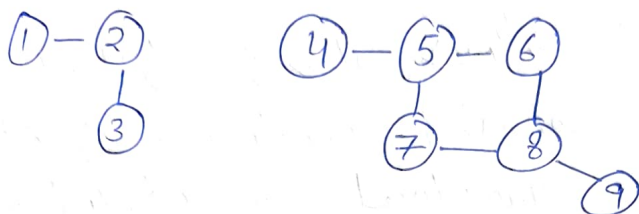
// to handle disconnected components.

```
    unordered_map <int, bool> visited;
    for (int i=0; i<n; i++) {
        if (!visited[i]) {
            bool ans = isCycleBFS(i,
                visited, adj);
            if (ans == 1)
                return "YES";
        }
    }
```

```
    return "NO";
}
```

NOTE : if node is visited & is parent then neglect.

DFS



same condition

if (visited && not parent)
→ cycle present.

code

```
bool isCyclicDFS (int node, int
unordered_map <int, bool> & visited,
unordered_map <int, bool> & adj) {
    visited [node] = true;
```

```
    for (auto neighbour : adj [node]) {
        if (!visited [neighbour]) {
            bool isCyclicDFS CycleDetected
```

```
= isCyclicDFS (neighbour, node,
visited, adj);
```

```
    if (cycleDetected)
        return true;
```

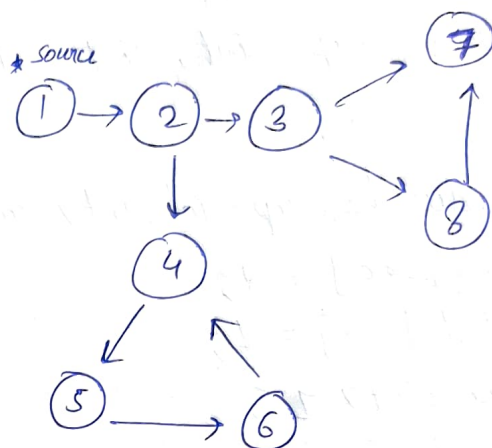
```
    }
```

```
    else if (neighbour != parent) {
        // cycle present
        return true;
    }
```

```
    } return false;
}
```

lec 89

Cycle Detection in Directed Graph



① Using BFS

Adj list

```
1 → 2
2 → 3, 4
3 → 7, 8
4 → 5
5 → 6
6 → 4
7 → ∅
8 → 8
```

visited array (map <int, bool>)

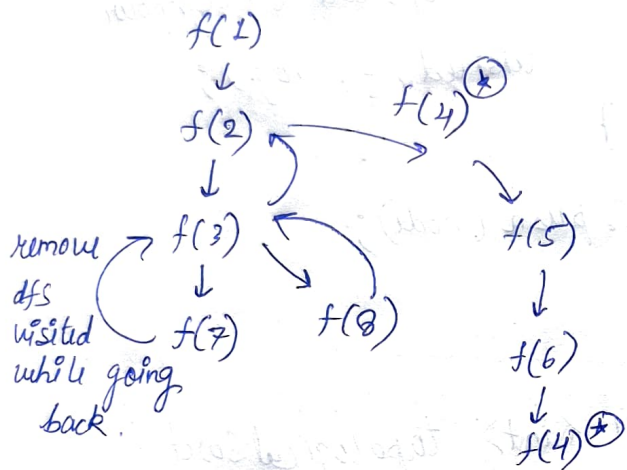
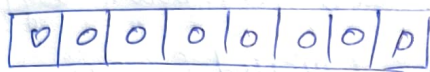
0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8

we can't use logic of undirected graph (visited && not parent) because

→ ③ → ⑦ because here 7 must be unvisited. when checking for ⑧.

we will make a new array dfs visited to keep track for dfs calling.

dfs visited (map <int, bool>)



```
if (visited[node] == true &&
    dfs_visited[node] == true) {
    cycle is present;
}
}
```

code

```
bool checkDFS Cycle (node, &visited,
    &dfs_visited, &adj) {
```

```
    visited[node] = node;
```

```
    dfs_visited[node] = node;
```

```
    for (auto: neighbour & adj[node]) {
```

```
        if (!visited[neighbour]) {
```

```
            bool cycleDetected =
```

```
            checkDFS Cycle (neighbour, visited,
```

```
            &dfs_visited, &adj);
```

```
        if (cycleDetected)
```

```
            return true;
```

```
    }
```

```
    else if (dfs_visited[neighbour]) {
```

```
        return true;
```

```
    }
```

```
}
```

```
dfs_visited = false;
```

```
return false;
```

```
}
```

```
int Detect Cycle (int n, & edges) {
    // create adj list (directed)
```

```
    unordered_map <int, bool> visited;
    // dfs_visited;
```

```
    for (int i=1; i<=n; i++) {
```

```
        if (!visited[i]) {
```

```
            bool Cycle Found = check(DFS cycle) Cycle DFS
                (i, visited, dfs_visited, adj);
```

```
            if (Cycle Found)
```

```
                return true;
```

```
        }
```

```
    return false;
```

```
}
```

lc 90

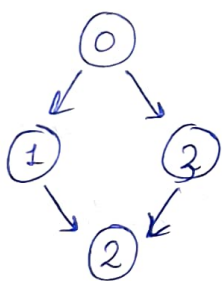
Topological Sort

using DFS

DAG $\rightarrow$ Directed Acyclic graph

$\rightarrow$ Linear ordering of vertices such that for every edge $u \rightarrow v$, u always appears before v in that ordering.

e.g



Adj list
 $0 \rightarrow 1, 2$
 $1 \rightarrow 2$
 $2 \rightarrow \phi$
 $3 \rightarrow 2$

Topological order $\rightarrow 0 \ 1 \ 3 \ 2$
 (valid) $\checkmark$

$1 \ 3 \ 2 \ 0 \rightarrow$ invalid

Multiple orders can be valid for single graph.

#approach - DFS + stack (LIFO)

NOTE: No valid order for cyclic graph.

we visit node in DFS order and we push node when we backtrack.

valid topoSort (int node, map<int, vector<int>> adj, stack<int> s, map<int, bool> visited) {

visited[node] = 1;

for (auto neighbour : adj[node])

if (!visited[neighbour])

topoSort (neighbour,

visited, s, adj);

}

s.push(node);

}

vector<int> topologicalSort (

vector<vector<int>> &edges, int v, e)

// create adj list

map<int, list<int>> adj;

for (int i=0; i<e; i++){

int u = edges[i][0];

int v = edges[i][1];

adj[u].push_back(v);

} map<int, bool> visited;

stack<int> s;

// call dfs topological sort

for (int i=0; i<v; i++){

if (!visited[i])

topoSort (i, visited, s, adj)

}

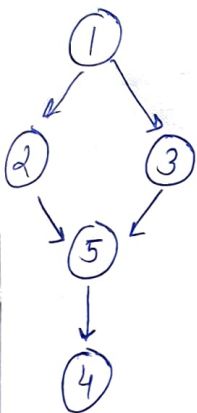
vector<int> ans;

```

while (!s.empty()) {
    ans.push_back(s.top());
    s.pop();
}
return ans;
}

```

Q1 Topological Sort using Kahn's Algorithm (BFS)



Adj list

1 → 2, 3

2 → 5

3 → 5

4 → ∅

5 → 4

approach

- ① Find indegree of all nodes.
- ② Put '0' - indegree nodes in queue
- ③ Do BFS

NOTE: Jis node ko karta rhi ho (pop) uske baad update indegree & then push '0' indegree nodes in queue.

code

```

vector<int> Topological Sort {
    vector<vector<int>> &edges, int v, e);
    unordered_map<int, list<int>> adj;
    for (int i=0; i<v; i++){
        // find all indegrees
        vector<int> indegree(v);
        for (auto i: adj) {
            for (auto j: i.second) {
                indegree[j]++;
            }
        }
        // 0 indgr push in queue
        queue<int> q;
        for (int i=0; i<v; i++) {
            if (indegree[i]==0) {
                q.push(i);
            }
        }
        // do BFS
        vector<int> ans;
        while (!q.empty()) {
            int front = q.front();
            q.pop();
            // ans store
            ans.push_back(front);
        }
    }
}

```

// neighbour indeg update

```
for (auto neighbour : adj [front])
{
    indegree [neighbour]--;
    if (indegree [neighbour] == 0)
        q.push (neighbour);
}
```

} return ans;

} T.C

T.C = $O(N+E)$

S.C = $O(N+E)$.

lec 92 Cycle Detection in a directed graph using BFS

(reusing Kahn's algorithm).

same code only change:

// making adj list

```
for (int i=0; i<edges.size(); i++)
{
    int u = edges[i][0]-1;
    int v = edges[i][1]-1;
    adj[u].push_back(v);
}
```

// do bfs

int cnt = 0;

while (!q.empty()) {

int front = q.front();

q.pop();

cnt++;

for (auto neighbour : adj [front])

{

same code

}

}

if (cnt == n)

return false;

else

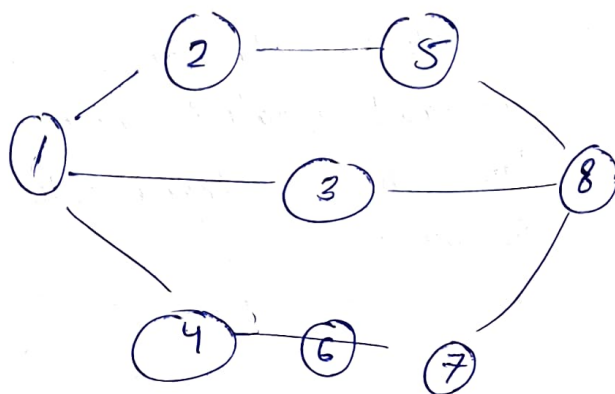
return true;

}

Shortest Path in Undir Graph

(from s $\rightarrow$ t)

BFS



shortest path

to (1,8) = [1, 3, 8]

approach : BFS ($s \rightarrow t$)

① make adj list & visited array

1 $\rightarrow$ 2, 3, 4 } parent.
2 $\rightarrow$ 1, 5
3 $\rightarrow$ 1, 8
4 $\rightarrow$ 1, 6
5 $\rightarrow$ 2, 8
6 $\rightarrow$ 4, 7
7 $\rightarrow$ 6, 8
8 $\rightarrow$ 3, 5, 7

visited

0	0	0	0	0	0	0	0
1	2	3	4	5	6	7	8

Parent

-1	1	1	1	2	4	6	3
1	2	3	4	5	6	7	8

this is useful to output our ans (backtrack from destination node to source node).

code

vector<int> shortestPath(
vector edges, int n, m, s, t) {

// create adj list (undir)

// do bfs.

un_map<int, bool> visited;

un_map<int, int> parent;

queue<int> q;

q.push(s); parent[s] = -1;

visited[s] = true;

while (!q.empty()) {

int front = q.front();

q.pop();

for (auto i: adj[front]) {

if (!visited[i]) {

visited[i] = true;

parent[i] = front;

q.push(i);

}

}

}

// prepare shortest path

vector<int> ans;

int currentNode = t;

while (

ans.push_back(t);

while (currentNode != s) {

currentNode = parent[currentNode];

ans.push_back(currentNode);

} reverse(ans.begin(), ans.end());
return ans;

}

T.C = $O(N+E)$

S.L = $O(N+E)$.